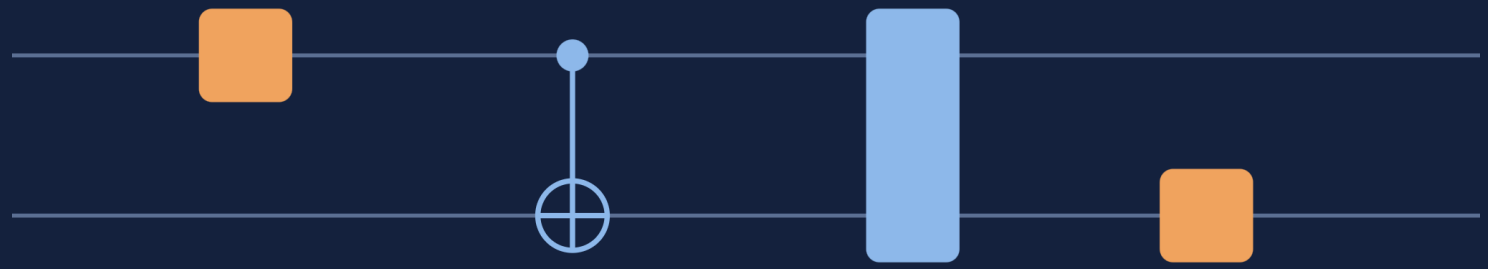
Circuitos cuánticos

Un simulador de qubits en Python con el patrón Command: estado de Bell, teleportación y algoritmo de Grover, publicado como librería en PyPI.



Carlos Andrés Gómez Vasco· Desarrollo de Software

AGENDA

Ocho etapas en 120 minutos

Inicio · duración	Etapa	Entregable
0:00 · 15 min	1. Física: qubits, medición y puertas	Vector de estado
0:15 · 15 min	2. Diseño: el patrón Command	Diagrama de clases
0:30 · 15 min	3. Modelo: EstadoCuantico	Amplitudes y medición
0:45 · 20 min	4. Puertas y circuito	Paso a paso y deshacer
1:05 · 15 min	5. Bell, teleportación y Grover	Tres algoritmos
1:20 · 10 min	6. Interfaz gráfica en tkinter	GUI.py
1:30 · 5 min	7. Pruebas con pytest	tests/test_core.py
1:35 · 20 min	8. Publicar en TestPyPI y PyPI	pip install circuitos-cuanticos
1:55 · 5 min	Cierre	Retos

ETAPA 1 · FÍSICA · 15 MIN

Un qubit: dos amplitudes complejas

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

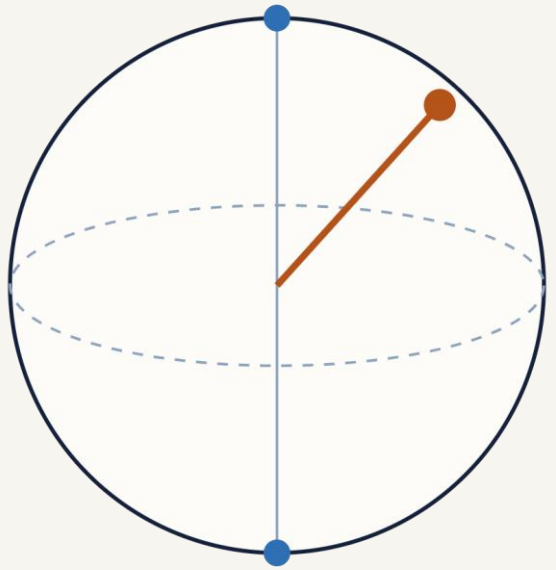
α, β complejos con $|\alpha|^2 + |\beta|^2 = 1$

Superposición

Si α y β son ambos distintos de cero, el qubit no está «en 0 o en 1».

Regla de Born

Al medir sale 0 con probabilidad $|\alpha|^2$ y 1 con $|\beta|^2$; el estado colapsa.



Esfera de Bloch: $|0\rangle$ arriba, $|1\rangle$ abajo; cada punto de la superficie es un qubit.

ETAPA 1 · FÍSICA · 15 MIN

Varios qubits: el vector crece como 2^n

TAMAÑO DEL VECTOR DE ESTADO

Qubits	Amplitudes 2^n	Memoria
1	2	32 B
2	4	64 B
10	1 024	16 KB
20	1 048 576	16 MB
30	≈ 1 073 millones	16 GB

Cada amplitud compleja ocupa 16 bytes. Por eso simular es fácil con pocos qubits e imposible con muchos.

Orden de los qubits

En |q0 q1 q2⟩ el qubit 0 va a la izquierda: |110⟩ es el índice 6 del vector.

Entrelazamiento

(|00⟩ + |11⟩)/√2 no se puede escribir como «estado de q0» por «estado de q1».

ETAPA 1 · FÍSICA · 15 MIN

Las puertas son matrices unitarias

Hadamard H

$$\frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\ 1 & -1\end{bmatrix}$$

$|0\rangle \to (|0\rangle+|1\rangle)/\sqrt{2}$: crea superposición.

NOT cuántico X

$$\begin{bmatrix}0 & 1\\ 1 & 0\end{bmatrix}$$

Intercambia $|0\rangle$ y $|1\rangle$.

Fase Z

$$\begin{bmatrix}1 & 0\\ 0 & -1\end{bmatrix}$$

Cambia el signo de $|1\rangle$.

CNOT

$$\begin{bmatrix}1 & 0 & 0 & 0\\ 0 & 1 & 0 & 0\\ 0 & 0 & 0 & 1\\ 0 & 0 & 1 & 0\end{bmatrix}$$

Niega el objetivo si el control vale 1.

Todas son unitarias: $U^\dagger U = I$. Por eso cualquier puerta se deshace aplicando $U^\dagger$.

La medición es la excepción: no es unitaria y no se puede revertir.

ETAPA 2 · DISEÑO · 15 MIN

Separar la lógica de la interfaz



core.py nunca importa tkinter. Se usa igual desde la consola, un notebook o las pruebas.

Sin Qiskit. Sólo NumPy: los estudiantes ven toda la física.

Misma estructura que el proyecto de osciladores.

ETAPA 2 · DISEÑO · 15 MIN

Un circuito es una lista de órdenes

SIN PATRÓN

```
def ejecutar(estado, lista):
    for nombre, qubits in lista:
        if nombre == "H":
            ...
        elif nombre == "CNOT":
            ...
        elif nombre == "Medir":
            ...
# ¿deshacer? ¿paso a paso? ¿dibujar?
```

Las puertas son texto: nadie recuerda qué se hizo ni cómo volver atrás.

CON COMMAND

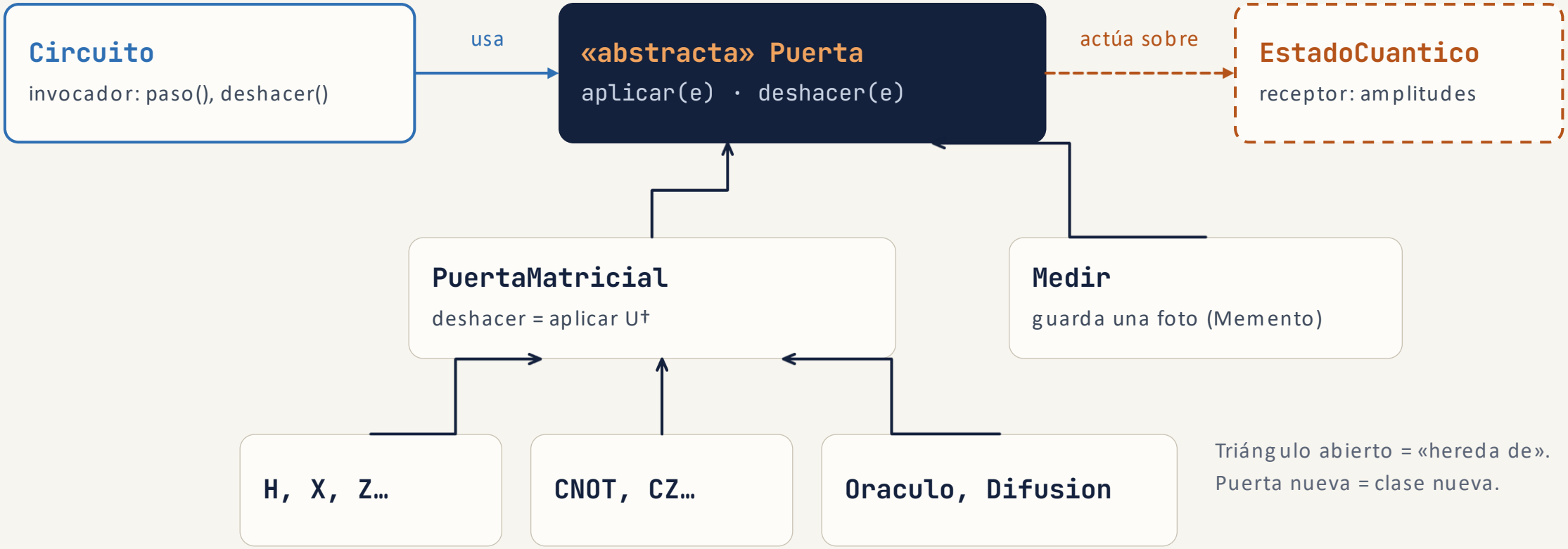
```
c = Circuito(2)
c.agregar(H(0)).agregar(CNOT(0, 1))

c.paso()        # aplica H
c.paso()        # aplica CNOT
c.deshacer()    # revierte CNOT
# cada puerta sabe hacerse y deshacerse
```

Cada puerta es un objeto que sabe aplicarse y deshacerse.

ETAPA 2 · DISEÑO · 15 MIN

Patrón Command: cada puerta es una orden



ETAPA 2 · DISEÑO · 15 MIN

¿Por qué Command y no otra cosa?

Paso a paso

Aplicar una puerta y mirar el estado: ideal para enseñar.

Deshacer

U† recorre el camino de vuelta: la física es reversible.

Circuito = datos

La lista se puede dibujar, guardar y contar.

Extensible

Una puerta nueva es una clase nueva; nada más cambia.

ALTERNATIVAS DESCARTADAS

Strategy

Sirve para elegir UN algoritmo; aquí hay una secuencia de operaciones.

Funciones sueltas

No recuerdan qué hicieron: no hay deshacer ni dibujo.

Una matriz total

Multiplicar todo de una vez pierde los pasos intermedios.

ETAPA 3 · MODELO · 15 MIN

Antes del código: el Python que vamos a leer

CÓDIGO	QUÉ SIGNIFICA
`0.6 + 0.8j`	Un número complejo en Python: la j es la unidad imaginaria i. Las amplitudes son complejas.
`np.abs(a) ** 2`	Módulo al cuadrado (** es potencia): la probabilidad de Born de la amplitud a.
`class Nombre: … self`	Un molde de objetos; dentro de sus métodos, self es «el objeto mismo».
`@abstractmethod`	Obliga a toda subclase a escribir ese método: así se define un contrato.
`v.reshape(2, 2, 2)`	Ver los mismos 8 números como un cubo de 2 × 2 × 2: un eje por qubit.
`return self`	Devolver el propio objeto permite encadenar: c.agregar(H(0)).agregar(X(1)).

ETAPA 3 · MODELO · 15 MIN

La clase EstadoCuantico

```
class EstadoCuantico:
    def __init__(self, n_qubits):
        self.n = n_qubits
        self._psi = np.zeros(2 ** n_qubits, dtype=complex)
        self._psi[0] = 1.0                          # |00…0⟩
    def aplicar_matriz(self, U, qubits):
        k = len(qubits)
        psi = self._psi.reshape([2] * self.n)       # un eje por qubit
        psi = np.moveaxis(psi, qubits, range(k))    # elegidos al frente
        forma = psi.shape
        psi = (U @ psi.reshape(2 ** k, -1)).reshape(forma)
        psi = np.moveaxis(psi, range(k), qubits)    # de vuelta a su lugar
        self._psi = psi.reshape(-1)
```

Nunca se construye la matriz de $2^n \times 2^n$: sólo se multiplica el bloque pequeño U.

ETAPA 3 · MODELO · 15 MIN

EstadoCuantico, paso a paso

CÓDIGO	QUÉ SIGNIFICA
`np.zeros(2 ** n, dtype=complex)`	Un vector de 2^n amplitudes complejas, todas en cero…
`self._psi[0] = 1.0`	…salvo la primera: el registro empieza en $\|00\ldots0\rangle$.
`reshape([2] * self.n)`	Los mismos números como tabla con un eje por qubit: psi[q0, q1, q2].
`np.moveaxis(…)`	Pone adelante los ejes de los qubits sobre los que actúa la puerta.
`U @ psi.reshape(2 ** k, -1)`	Multiplica U por todas las columnas a la vez: actúa igual sea cual sea el resto.

EN LA CONSOLA

```
>>> Hm = H(0).matriz()
>>> e = EstadoCuantico(2)
>>> e.aplicar_matriz(Hm, [1])
>>> e.ket()
0.707|00⟩ + 0.707|01⟩
>>> e.aplicar_matriz(Hm, [0])
>>> len(e.probabilidades())
4
```

ETAPA 3 · MODELO · 15 MIN

Medir: la regla de Born en código

```
def probabilidades(self):
    p = np.abs(self._psi) ** 2                          # regla de Born
    return {self.etiqueta(k): p[k] for k in range(p.size) if p[k] > 1e-12}
def medir(self, disparos=1000, semilla=None):
    rng = np.random.default_rng(semilla)
    p = np.abs(self._psi) ** 2
    resultados = rng.choice(p.size, size=disparos, p=p)
    conteo = np.bincount(resultados, minlength=p.size)
    return {self.etiqueta(k): int(c) for k, c in enumerate(conteo) if c > 0}
```

rng.choice

Un dado cargado: sale k con probabilidad p[k].

np.bincount

Cuenta cuántas veces salió cada resultado.

semilla

Fija el azar: misma semilla, mismos resultados.

ETAPA 4 · PUERTAS Y CIRCUITO · 20 MIN

Las puertas son comandos

```
class Puerta(ABC):                          # el COMANDO abstracto
    @abstractmethod
    def aplicar(self, estado): ...
    @abstractmethod
    def deshacer(self, estado): ...

class PuertaMatricial(Puerta):              # puertas unitarias
    def aplicar(self, estado):
        estado.aplicar_matriz(self.matriz(), self.qubits)
    def deshacer(self, estado):
        estado.aplicar_matriz(self.matriz().conj().T, self.qubits)

class H(PuertaMatricial):
    def matriz(self):
        return np.array([[1, 1], [1, -1]]) / np.sqrt(2)
```

U† = conj().T

La traspuesta conjugada. Para una matriz unitaria es su inversa.

Una clase por puerta

H, X, CNOT… sólo dicen cuál es su matriz; aplicar y deshacer se heredan.

ETAPA 4 · PUERTAS Y CIRCUITO · 20 MIN

Las puertas, en palabras

CÓDIGO	QUÉ SIGNIFICA
`class Puerta(ABC)`	Un contrato: toda orden debe saber aplicarse y deshacerse.
`PuertaMatricial`	Reúne lo común a las puertas unitarias, descritas por una matriz U.
`self.matriz().conj().T`	U† (conjugar y trasponer). Como U†U = I, aplicar U y luego U† deja el estado como estaba.
`CNOT(0, 1)`	Control = qubit 0, objetivo = qubit 1: niega el objetivo sólo si el control vale 1.
`Oraculo(n, marcado)`	Cambia el signo de una sola amplitud: la del estado buscado en Grover.
`Difusion(n)`	Refleja cada amplitud respecto al promedio: a_k → 2·media − a_k.

ETAPA 4 · PUERTAS Y CIRCUITO · 20 MIN

El circuito es el invocador

```
class Circuito:                                    # el INVOCADOR
    def __init__(self, n_qubits):
        self.puertas = []                          # lista de comandos
        self.estado = EstadoCuantico(n_qubits)     # el receptor
        self.posicion = 0                          # cuántos se han aplicado
    def agregar(self, puerta):
        self.puertas.append(puerta)
        return self                                # permite encadenar
    def paso(self):
        self.puertas[self.posicion].aplicar(self.estado)
        self.posicion += 1
    def deshacer(self):
        self.posicion -= 1
        self.puertas[self.posicion].deshacer(self.estado)
```

Versión resumida

El archivo real revisa los límites, devuelve True o False y avisa a los observadores.

ejecutar()

Repite paso() hasta aplicar todas las puertas pendientes.

ETAPA 4 · PUERTAS Y CIRCUITO · 20 MIN

El estado de Bell, paso a paso

posición 0

$|00\rangle$

→

paso(): H(0) · posición 1

$0.707|00\rangle + 0.707|10\rangle$

→

paso(): CNOT · posición 2

$0.707|00\rangle + 0.707|11\rangle$

CÓDIGO	QUÉ SIGNIFICA
`self.posicion`	Un marcador: cuántas puertas ya se aplicaron. En la GUI es la línea naranja.
`return self`	Devuelve el mismo circuito para poder escribir c.agregar(H(0)).agregar(CNOT(0, 1)).
`deshacer()`	Retrocede el marcador y le pide a esa puerta que se revierta: el circuito no sabe cómo, la puerta sí.

ETAPA 4 · PUERTAS Y CIRCUITO · 20 MIN

Medir: la única orden irreversible

```
class Medir(Puerta):
    def aplicar(self, estado):
        self._antes = estado.amplitudes      # la foto (Memento)
        p1 = estado.probabilidad_uno(self.qubits[0])
        self.resultado = int(self._rng.random() < p1)
        ...                                  # anula la otra rama
    def deshacer(self, estado):
        estado._reemplazar(self._antes)      # vuelve a la foto
```

En la física

Medir colapsa el estado; no existe U† que lo revierta.

En el código

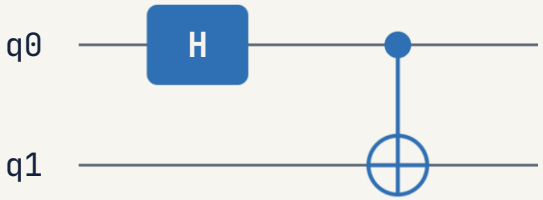
Por eso el comando guarda una copia antes de medir: patrón Memento.

En el laboratorio

Una medición real no se deshace. El simulador sí, porque guardó la foto.

ETAPA 5 · ALGORITMOS · 15 MIN

Estado de Bell: dos qubits, un solo destino



$$(|00\rangle + |11\rangle) / \sqrt{2}$$

1000 mediciones simuladas: 00 → 502 veces · 11 → 498 veces · 01 y 10 → nunca

Siempre iguales

Medir un qubit determina el otro: los resultados están correlacionados.

Sin estado propio

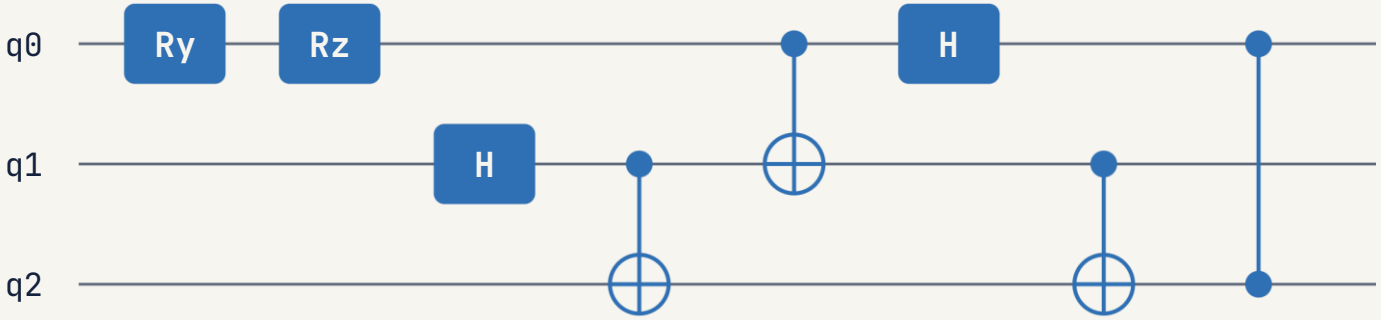
El vector de Bloch de cada qubit mide 0: el estado pertenece al par.

En código

```
Circuito(2)
.agregar(H(0))
.agregar(CNOT(0, 1))
```

ETAPA 5 · ALGORITMOS · 15 MIN

Teleportación: mover un estado sin moverlo



Comprobación (θ = 1, φ = 0.5)

Bloch de q0 al inicio

`(0.738, 0.403, 0.540)`

Bloch de q2 al final

`(0.738, 0.403, 0.540)`

1 • Preparar

RY(θ) y RZ(ϕ) dejan en q0 el estado que se quiere enviar.

2 • Medición de Bell

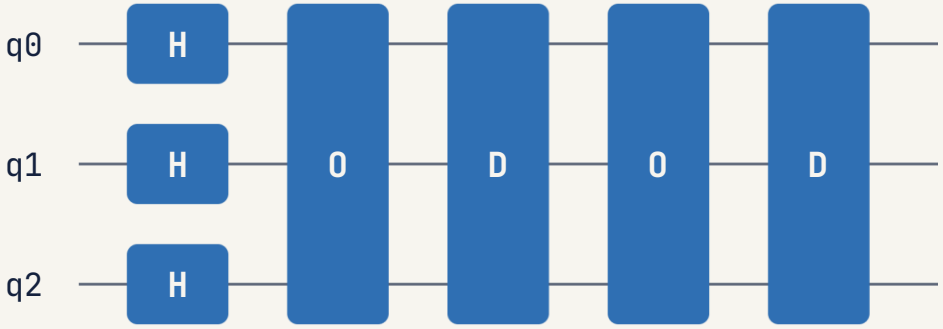
CNOT y H mezclan ese estado con el par entrelazado q1–q2.

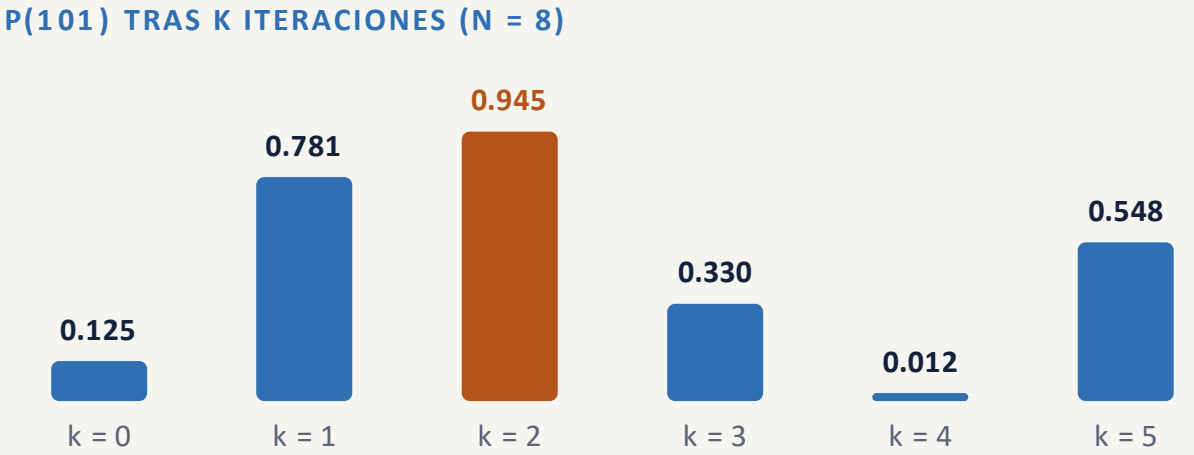
3 • Corregir

CNOT y CZ controladas reconstruyen el estado en q2.

ETAPA 5 · ALGORITMOS · 15 MIN

Grover: buscar en √N pasos





La idea

El oráculo marca con un signo menos; la difusión amplifica esa amplitud.

Cuántas veces

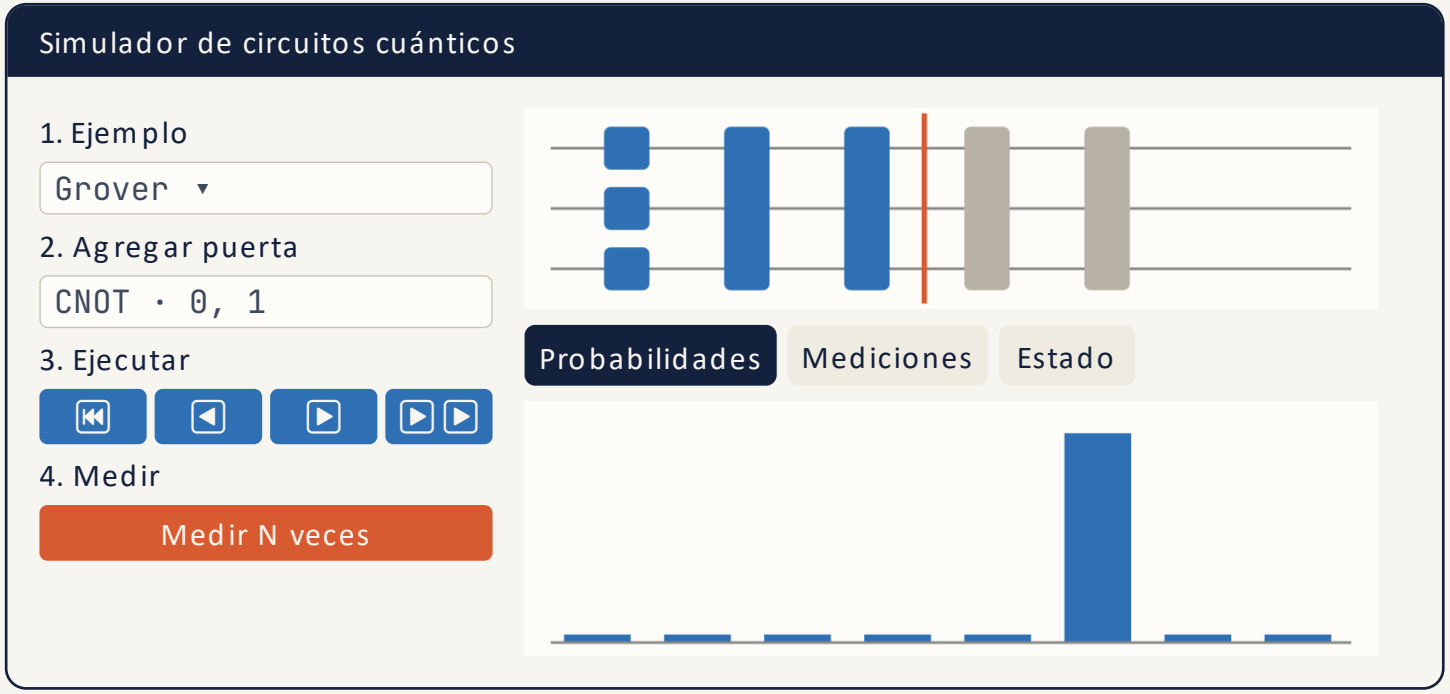
≈ (π/4)√N iteraciones: 2 para N = 8 y 25 para N = 1024.

Cuidado

Pasarse empeora: con 3 iteraciones la probabilidad cae a 0.330.

ETAPA 6 · INTERFAZ GRÁFICA · 10 MIN

La ventana: construir, avanzar y medir



Paso y deshacer

◀ y ▶ llaman a deshacer() y paso() del circuito.

Tres pestañas

Probabilidades exactas, mediciones simuladas y el ket con Bloch.

Circuito libre

Con «Circuito vacío» cada estudiante arma el suyo.

ETAPA 6 · INTERFAZ GRÁFICA · 10 MIN

La GUI, en lenguaje cotidiano

CÓDIGO	QUÉ SIGNIFICA
`circuito.suscribir(self._al_cambiar)`	Deja «el teléfono» de la GUI: tras cada paso o deshacer, la lógica llama a _al_cambiar y la ventana se redibuja.
`FabricaPuertas.crear(nombre, *qubits)`	Crea la puerta elegida en el menú. El asterisco reparte [0, 1] como dos argumentos: crear("CNOT", 0, 1).
`try: … except ValueError`	Si la puerta no tiene sentido (una CNOT con un solo qubit), muestra el mensaje en vez de cerrarse.
`isinstance(p, CNOT)`	Pregunta qué tipo de puerta es para dibujarla con su símbolo: punto de control y ⊕.
`i < c.posicion`	Las puertas ya aplicadas se pintan en azul y las pendientes en gris.

ETAPA 7 · PRUEBAS · 5 MIN

Las pruebas comprueban la física

```
def test_bell_solo_00_y_11():
    p = CatalogoCircuitos.bell().ejecutar().probabilidades()
    assert p == pytest.approx({"00": 0.5, "11": 0.5})

def test_deshacer_vuelve_al_estado_inicial():
    ...
    while c.deshacer():
        pass
    assert np.allclose(c.estado.amplitudes, EstadoCuantico(3).amplitudes)

$ pytest -v
================ 26 passed ================
```

Qué se prueba

- Bell y GHZ
- Teleportación con 4 estados
- Grover de 2 a 12 qubits
- U†U = I en cada puerta
- Deshacer y medir
- Born con 10 000 disparos

ETAPA 8 · PYPI · 20 MIN

La estructura que espera PyPI

```
circuitos-cuanticos/
├── pyproject.toml
├── README.md
├── LICENSE
├── src/
│   └── circuitos_cuanticos/
│       ├── __init__.py
│       └── core.py
├── tests/test_core.py
├── GUI.py            ← no se publica
└── ejemplo_consola.py
```

```
[project]
name = "circuitos-cuanticos"
version = "0.1.0"
requires-python = ">=3.9"
dependencies = ["numpy>=1.22"]

[project.optional-dependencies]
gui = ["matplotlib>=3.5"]

[tool.setuptools.packages.find]
where = ["src"]
```

Se instala como **circuitos-cuanticos** y se importa como **circuitos_cuanticos**. Sólo lo que está en src/ entra al paquete.

ETAPA 8 · PYPI · 20 MIN

Instalar el proyecto como un profesional

SI NO ESTÁ INSTALADA

```
PS> python .\GUI.py
No se encontró la librería 'circuitos_cuanticos'.
Desde la carpeta del proyecto, con el entorno
virtual activo, ejecute:
    python -m pip install -e ".[gui]"
```

GUI.py no busca la librería por su cuenta: avisa con un mensaje claro y se detiene. Así se trabaja profesionalmente.

LA SOLUCIÓN, UNA VEZ POR PROYECTO

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -e ".[gui,dev]"
python GUI.py
```

CÓDIGO	QUÉ SIGNIFICA
`.venv`	Un Python propio del proyecto, aislado de los demás.
`-e`	Enlace a src/: los cambios en core.py se ven al instante.
`[gui,dev]`	Añade matplotlib, pytest, build y twine.

En cada terminal nueva: activar el entorno · el prompt debe mostrar (.venv)

ETAPA 8 · PYPI · 20 MIN

Seis pasos para publicar

1

Nombre y versión

`editar pyproject.toml`

2

Construir

`python -m build`

3

Revisar

`python -m twine check dist/*`

4

Ensayar en TestPyPI

```
twine upload --repository
testpypi dist/*
```

5

Probar la instalación

```
pip install -i
https://test.pypi.org/simple/ …
```

6

Publicar

`python -m twine upload dist/*`

Usuario: __token__ · contraseña: el token de API · cada versión se sube una sola vez

ETAPA 8 · PYPI · 20 MIN

Paso 1: la información va en pyproject.toml

```
[project]
name = "circuitos-cuanticos-sunombre"     # URL y pip install
version = "0.1.0"                         # cambia en cada envío
description = "Simulador de circuitos"    # subtítulo de la página
readme = "README.md"                      # cuerpo de la página
authors = [{ name = "Su Nombre", email = "su@correo.com" }]
license = "MIT"
dependencies = ["numpy>=1.22"]            # pip las instala solo

[project.urls]
Homepage = "https://github.com/su_usuario/circuitos-cuanticos"
```

Mismo archivo, dos destinos

TestPyPI y PyPI leen el mismo pyproject.toml. Sólo cambia el comando twine.

Versión en dos lugares

version en pyproject.toml y __version__ en __init__.py deben decir lo mismo.

Archivo: circuitos-cuanticos/pyproject.toml · se edita con cualquier editor de texto

ETAPA 8 · PYPI · 20 MIN

Pasos 2 y 3: construir y revisar en consola

```
# 0. Ubicarse en el proyecto y activar el entorno
cd C:\Users\Usuario\proyectos\circuitos-cuanticos
.\.venv\Scripts\Activate.ps1

# 1. Borrar construcciones anteriores
Remove-Item -Recurse -Force dist -ErrorAction SilentlyContinue

# 2. Construir el paquete
python -m build

# 3. Revisar que PyPI lo aceptará
python -m twine check dist/*
```

RESULTADO EN LA CARPETA DIST/

CÓDIGO	QUÉ SIGNIFICA
`….tar.gz`	Código fuente (sdist).
`….whl`	La rueda (wheel): lo que pip instala.
`PASSED`	twine check aprobó los metadatos y el README.

Todos los comandos se escriben en PowerShell, dentro de la carpeta del proyecto, con (.venv) activo

ETAPA 8 · PYPI · 20 MIN

Antes de subir: dos cuentas y dos tokens

	TestPyPI · ensayo	PyPI · real
Sitio	test.pypi.org	pypi.org
Cuenta	Registro propio	Otro registro, independiente
Seguridad	2FA obligatoria	2FA obligatoria
Token	Account settings → API tokens	Account settings → API tokens
Alcance	1.ª vez: Entire account	1.ª vez: Entire account

DÓNDE SE GUARDAN LOS TOKENS

C:\Users\Usuario\.pypirc

```
[testpypi]
  username = __token__
  password = pypi-AgENdGVzdC5weXBp...

[pypi]
  username = __token__
  password = pypi-AgEIcHlwaS5vcmc...
```

Cada token va en su sección. Nunca suba este archivo a GitHub.

Los tokens de ejemplo están recortados: los reales son mucho más largos

ETAPA 8 · PYPI · 20 MIN

Pasos 4 y 5: ensayo en TestPyPI

```
# 4. Subir al repositorio de ensayo
python -m twine upload --repository testpypi dist/*
#    → https://test.pypi.org/project/circuitos-cuanticos-sunombre/

# 5. Probar en un entorno limpio, en otra carpeta
python -m venv C:\temp\prueba
C:\temp\prueba\Scripts\Activate.ps1
python -m pip install `
    --index-url https://test.pypi.org/simple/ `
    --extra-index-url https://pypi.org/simple/ `
    circuitos-cuanticos-sunombre
python -c "import circuitos_cuanticos as cc; print(cc.__version__)"
```

--repository testpypi

Toma el token de la sección [testpypi] de .pypirc.

--extra-index-url

NumPy no está en TestPyPI: pip lo descarga de PyPI.

El acento grave `

Continúa el comando en la línea siguiente en PowerShell.

TestPyPI se puede usar todas las veces que haga falta; también exige una versión nueva en cada subida

ETAPA 8 · PYPI · 20 MIN

Paso 6: la versión final en PyPI

```
# Volver al entorno del proyecto
deactivate
cd C:\Users\Usuario\proyectos\circuitos-cuanticos
.\.venv\Scripts\Activate.ps1

# 6. Subir los MISMOS archivos de dist/ a PyPI
python -m twine upload dist/*
#    → https://pypi.org/project/circuitos-cuanticos-sunombre/

# Desde ahora, cualquier persona puede instalarlo
pip install "circuitos-cuanticos-sunombre[gui]"
```

Para la siguiente versión

1. Subir la versión en pyproject.toml y __init__.py
2. Borrar dist/
3. python -m build y twine check
4. Subir a TestPyPI y probar
5. Subir a PyPI

Sin vuelta atrás

PyPI no acepta dos veces la misma versión, ni siquiera después de borrarla.

Sin --repository, twine sube a PyPI · el import sigue siendo circuitos_cuanticos

CIERRE · 5 MIN

Retos para la próxima clase

1. Deutsch-Jozsa

Decidir si una función es constante o balanceada con una sola consulta.

2. Ruido

Una puerta que aplica X al azar con probabilidad p: ¿cómo se degrada Bell?

3. Fourier cuántica

Construir la QFT con H y fases controladas: la base del algoritmo de Shor.

4. Otra interfaz

Un notebook de Jupyter o una línea de comandos, sin tocar core.py.

Cada reto se entrega como una nueva versión del paquete: 0.2.0, 0.3.0…